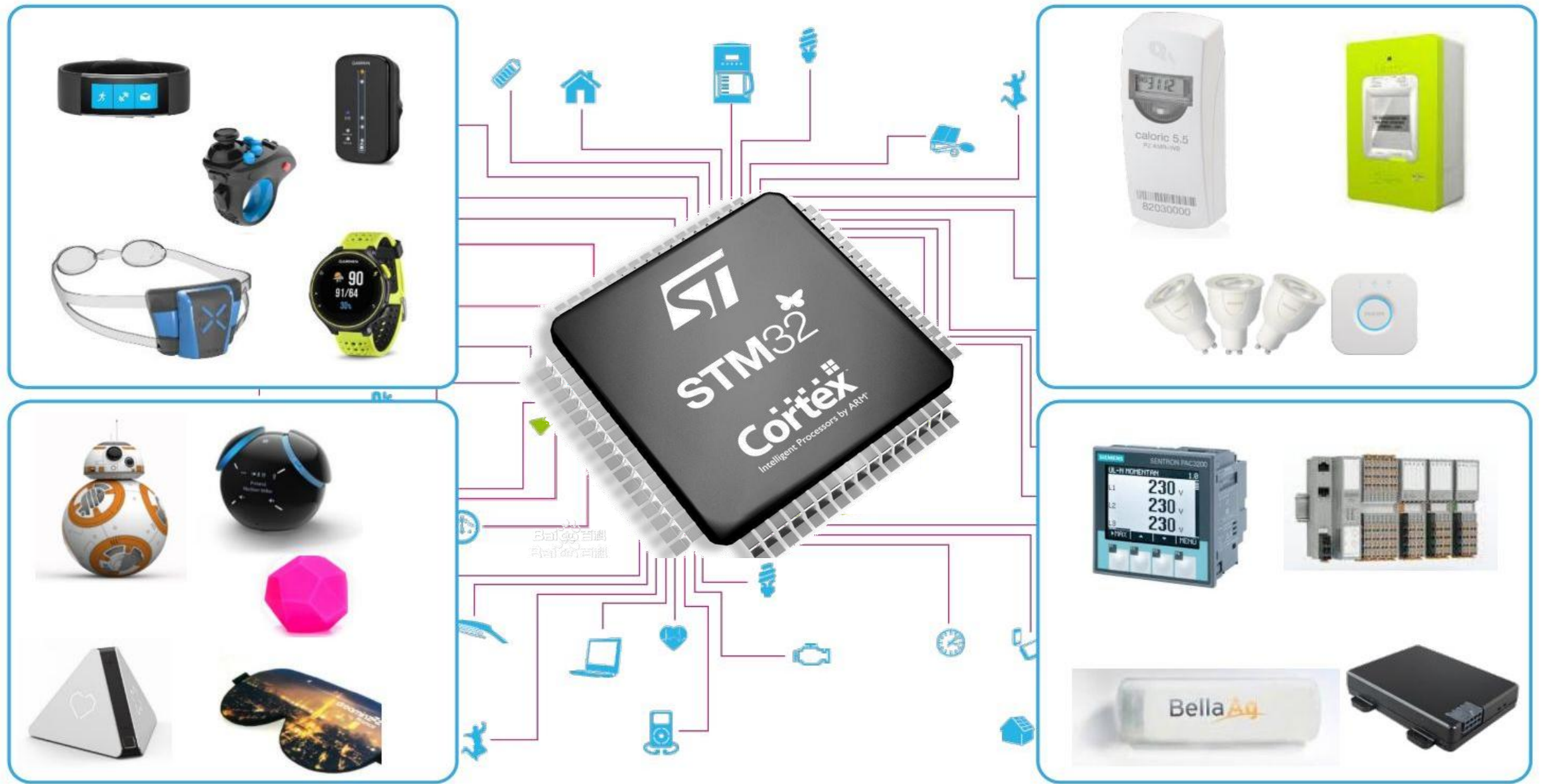




Instructor : WangXiaojing

Email : wangxj@sustech.edu.cn



Lecture6

TIMER-I

Topics

- Overview of Timer
- Basic Timer
- Overview of General Timer
- General Timer
 - Timing & Counting (Clock Selection)

Part I : Overview of Timer



00:00 / 00:59

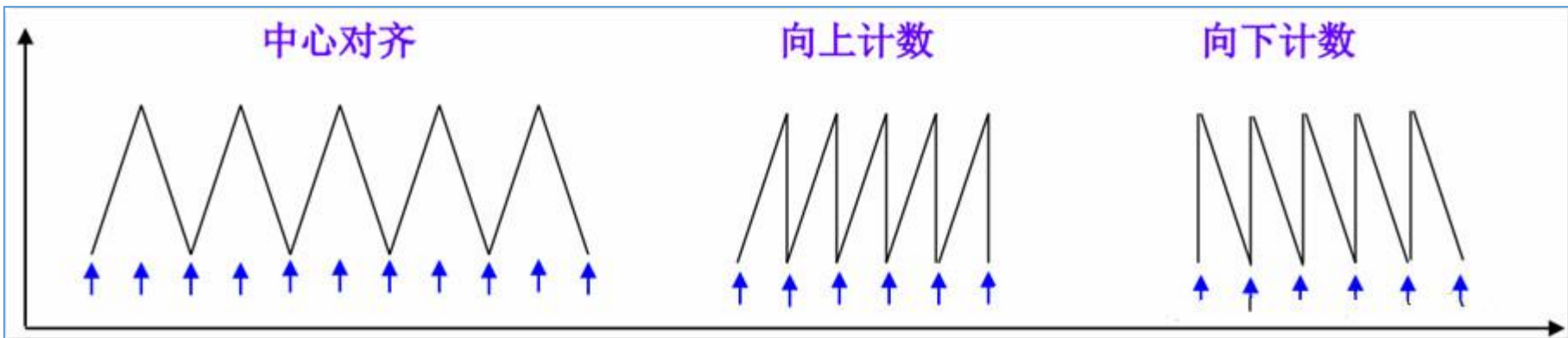
倍速



Counter Modes

- **Upcounting mode** : In upcounting mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register), then restarts from 0 and generates a counter overflow event.
- **Downcounting mode**: In downcounting mode, the counter counts from the auto-reload value (content of the TIMx_ARR register) down to 0, then restarts from the auto-reload value and generates a counter underflow event.
- **Center-aligned mode** (up/down counting): In center-aligned mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register) - 1, generates a counter overflow event, then counts from the auto-reload value down to 1 and generates a counter underflow event. Then it restarts counting from 0.

	定时器	计数器分辨率	计数器类型	预分频系数	产生DMA	捕获/比较通道	互补输出
高级定时器	TIM1	16位	向上/向下	1~65535	可以	4	有
	TIM8	16位	向上/向下	1~65535	可以	4	有
通用定时器	TIM2	16位	向上/向下	1~65535	可以	4	没有
	TIM3	16位	向上/向下	1~65535	可以	4	没有
	TIM4	16位	向上/向下	1~65535	可以	4	没有
	TIM5	16位	向上/向下	1~65535	可以	4	没有
基本定时器	TIM6	16位	向上	1~65535	可以	0	没有
	TIM7	16位	向上	1~65535	可以	0	没有



向上计数模式：计数器从0计数到自动加载值(TIMx_ARR)，然后重新从0开始计数并且产生一个计数器溢出事件。

向下计数模式：计数器从自动加载值(TIMx_ARR)开始向下计数到0，然后从自动装入的值重新开始，并产生一个计数器向下溢出事件。

中央对齐模式：计数器从0开始计数到(TIMx_ARR - 1)，产生一个计数器溢出事件，然后从重装值(TIMx_ARR)向下计数到1并且产生一个计数器溢出事件。之后计数器再从0开始计数。

- Counter
- Counter Mode
- Clock of the Counter (*CK_CNT*)
- The auto-reload value--
 - content of the *TIMx_ARR* register

Table 2. STM32F103xC, STM32F103xD and STM32F103xE features and peripheral counts

Peripherals		STM32F103Rx			STM32F103Vx			STM32F103Zx		
Flash memory in Kbytes		256	384	512	256	384	512	256	384	512
SRAM in Kbytes		48	64		48	64		48	64	
FSMC		No			Yes ⁽¹⁾			Yes		
Timers	General-purpose	4								
	Advanced-control	2								
	Basic	2								

Table 4. High-density timer feature comparison

Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/compare channels	Complementary outputs
TIM1, TIM8	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	Yes
TIM2, TIM3, TIM4, TIM5	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	No
TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No

Advanced Timers

General Timers

Basic Timers

Part II : Basic Timer

17.1 TIM6 and TIM7 introduction

The basic timers TIM6 and TIM7 consist of a 16-bit auto-reload counter driven by a programmable prescaler.

They may be used as generic timers for time-base generation but they are also specifically used to drive the digital-to-analog converter (DAC). In fact, the timers are internally connected to the DAC and are able to generate a TRGO signal.

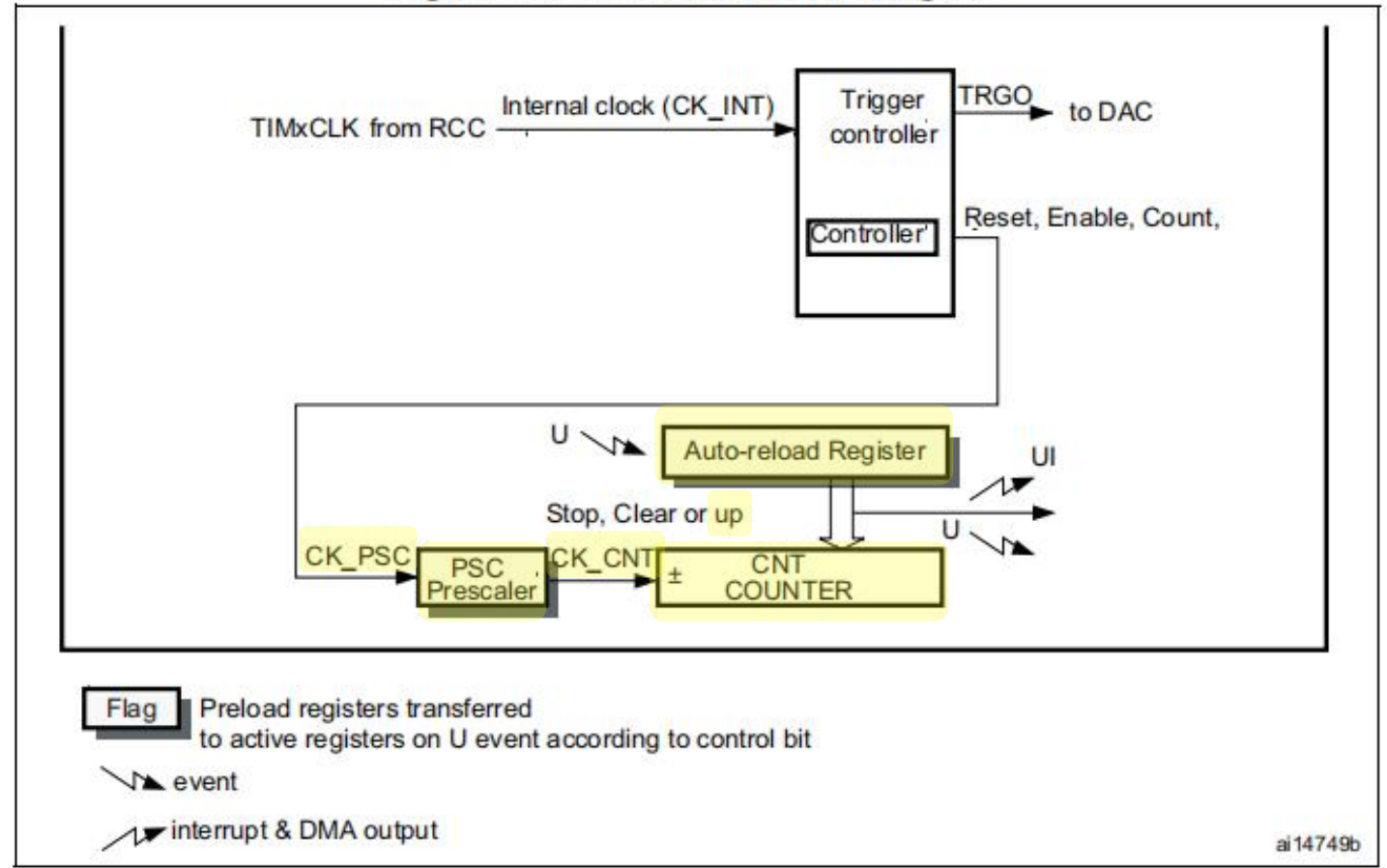
The timers are completely independent.

17.2 TIM6 and TIM7 main features

Basic timer (TIM6 and TIM7) features:

- 16-bit auto-reload upcounter
- 16-bit programmable prescaler frequency by any factor between 1 and 65535
- Synchronization circuit to trigger the timer
- Interrupt/DMA generation on the timer

Figure 169. Basic timer block diagram



17.3 TIM6 and TIM7 functional description

17.3.1 Time-base unit

The main block of the programmable timer is a 16-bit upcounter with its related auto-reload register. The counter clock can be divided by a prescaler.

The counter, the auto-reload register and the prescaler are controlled by software. This is true even when the counter is running.

The time-base unit includes:

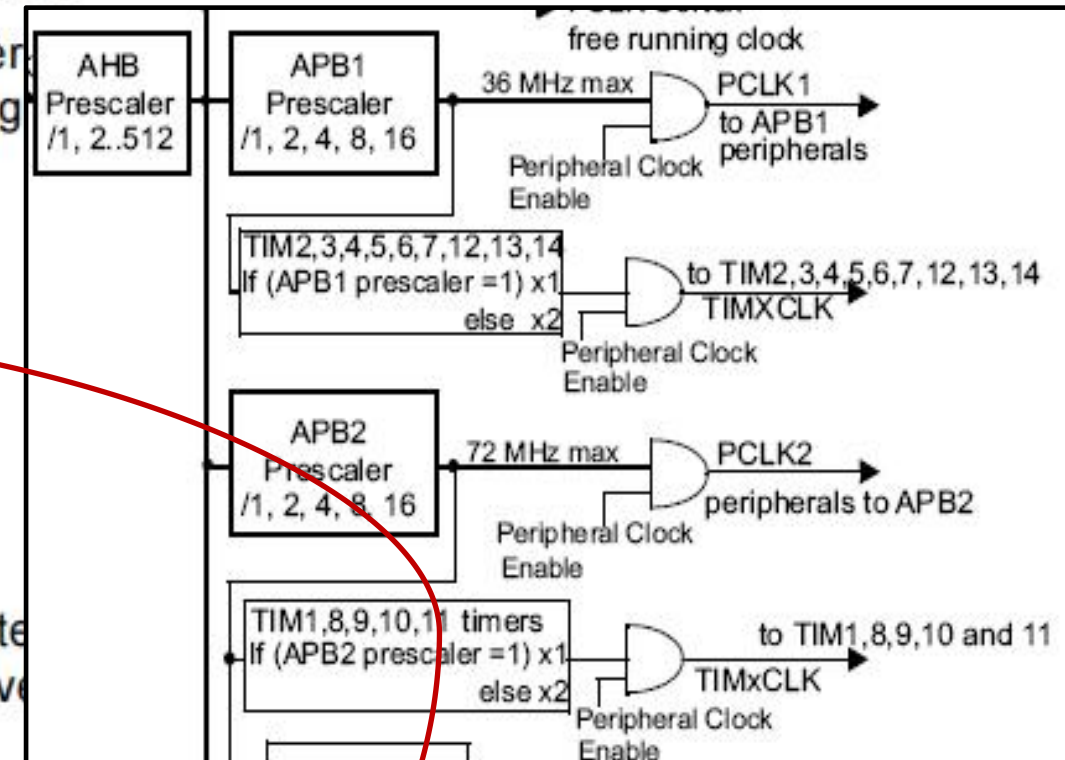
- Counter register (TIMx_CNT)
- Prescaler register (TIMx_PSC)
- Auto-Reload register (TIMx_ARR)

17.3.2 Counting mode

The counter counts from 0 to the auto-reload value (content of TIMx_ARR) then restarts from 0 and generates a counter overflow event.

17.3.3 Clock source

The counter clock is provided by the Internal clock (CK_INT) source.



17.4 TIM6 and TIM7 registers

17.4.1 TIM6 and TIM7 control register 1 (TIMx_CR1)

CEN: Counter enable

17.4.2 TIM6 and TIM7 control register 2 (TIMx_CR2)

17.4.3 TIM6 and TIM7 DMA/Interrupt enable register (TIMx_DIER)

UIE : Update interrupt enable)

17.4.4 TIM6 and TIM7 status register (TIMx_SR)

UIF : Update interrupt flag

17.4.5 TIM6 and TIM7 event generation register (TIMx_EGR)

UG : Update generation

17.4.6 TIM6 and TIM7 counter (TIMx_CNT)

17.4.7 TIM6 and TIM7 prescaler (TIMx_PSC)

17.4.8 TIM6 and TIM7 auto-reload register (TIMx_ARR)

CNT[15:0]: Counter value

PSC[15:0]: Prescaler value

The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.

ARR[15:0]: Auto-reload value

ARR is the value to be loaded into the actual auto-reload register.

time for cnt from X-->X+1

$(psc+1)/fck_psc$

time for cnt from 0-->ARR

$(ARR+1)*(psc+1)/fck_psc$

```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct)
```

```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct)
```

```
51 typedef struct
52 {
53     uint16_t TIM_Prescaler;      /*!< Specifies the prescaler value used to divide the TIM clock.
54                                   This parameter can be a number between 0x0000 and 0xFFFF */
55
56     uint16_t TIM_CounterMode;    /*!< Specifies the counter mode.
57                                   This parameter can be a value of @ref TIM_Counter_Mode */
58
59     uint16_t TIM_Period;         /*!< Specifies the period value to be loaded into the active
60                                   Auto-Reload Register at the next update event.
61                                   This parameter must be a number between 0x0000 and 0xFFFF. */
62
63     uint16_t TIM_ClockDivision;  /*!< Specifies the clock division.
64                                   This parameter can be a value of @ref TIM_Clock_Division_CKD */
65
66     uint8_t TIM_RepetitionCounter; /*!< Specifies the repetition counter value. Each time the RCR downcounter
67                                   reaches zero, an update event is generated and counting restarts
68                                   from the RCR value (N).
69                                   This means in PWM mode that (N+1) corresponds to:
70                                   - the number of PWM periods in edge-aligned mode
71                                   - the number of half PWM period in center-aligned mode
72                                   This parameter must be a number between 0x00 and 0xFF.
73                                   @note This parameter is valid only for TIM1 and TIM8. */
74 } TIM_TimeBaseInitTypeDef;
75
```

```
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState);
```

```
FlagStatus TIM_GetFlagStatus(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

```
void TIM_ClearFlag(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState);
```

```
ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

```
void TIM_ClearITPendingBit(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```



Basic timer--Timing Configuration

1、 Enable timer clock

`RCC_APB1PeriphClockCmd();`

2、 Initialization the time base structure

`TIM_TimeBaseInit();`

3、 Timer interrupt configuration

NVIC Configuration

`TIM_ClearFlag();`
`void TIM_ITConfig();`

4、 Enable timer

`TIM_Cmd();`

5、 Write interrupt service function.

`TIMx_IRQHandler();`

0x4000 4000 - 0x4000 43FF	Reserved	APB1
0x4000 3C00 - 0x4000 3FFF	SPI3/I2S	
0x4000 3800 - 0x4000 3BFF	SPI2/I2S	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 2400 - 0x4000 27FF	Reserved	
0x4000 2000 - 0x4000 23FF	TIM14 timer	
0x4000 1C00 - 0x4000 1FFF	TIM13 timer	
0x4000 1800 - 0x4000 1BFF	TIM12 timer	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	TIM5 timer	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	



Homework6-1

- Write out the registers those are used (configured or read) and **related with the Basic TIM** by the project (LED_BasicTimer) and their meanings one by one; ➡
- Program to realize: PA4 output 100Hz square wave;

Part III : Overview of General Timer

15.1 TIM2 to TIM5 introduction

The general-purpose timers consist of a 16-bit auto-reload counter driven by a programmable prescaler.

They may be used for a variety of purposes, including measuring the pulse lengths of input signals (*input capture*) or generating output waveforms (*output compare and PWM*).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

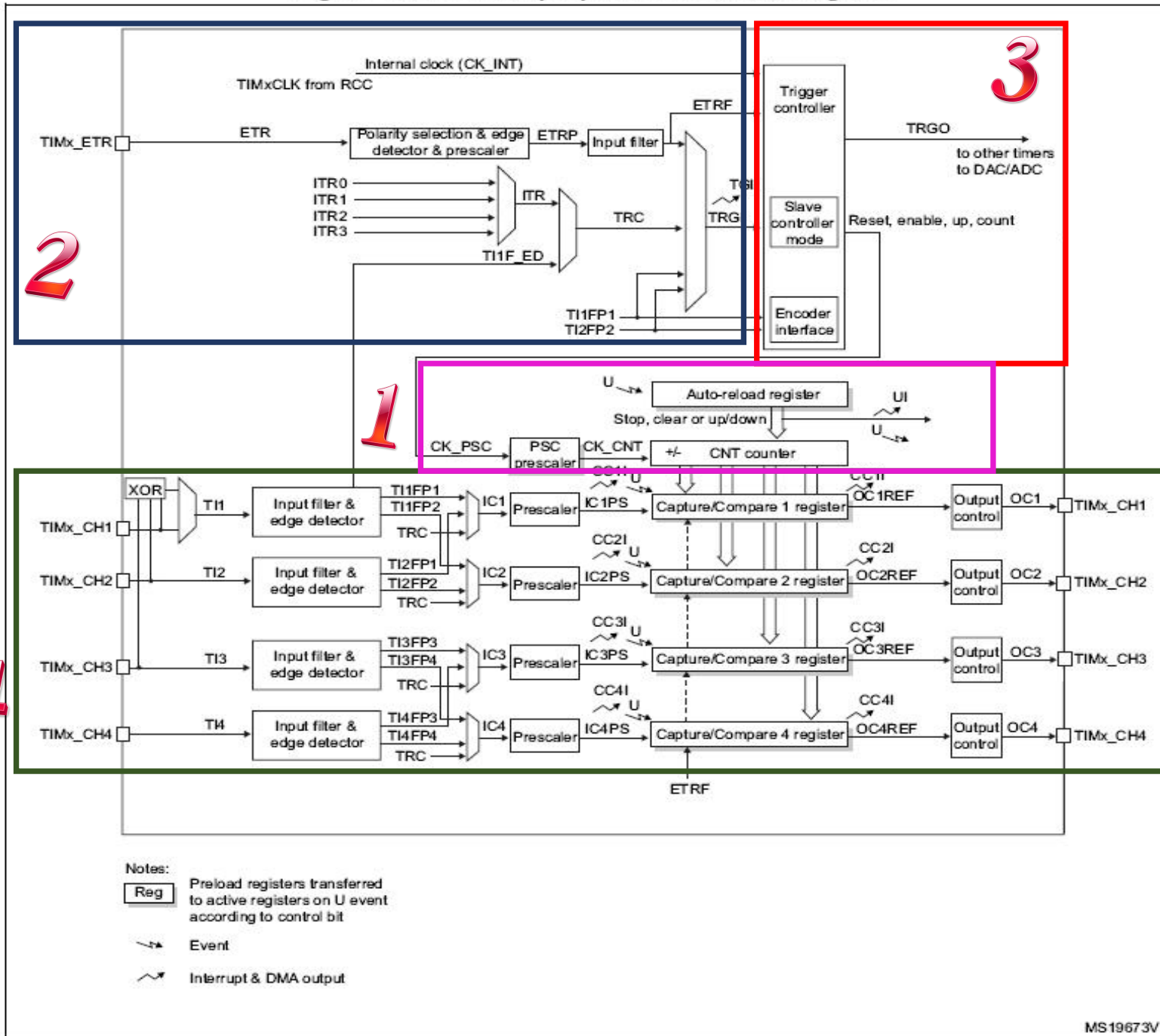
The timers are completely independent, and do not share any resources. They can be synchronized together as described in [Section 15.3.15](#).

15.2 TIMx main features

General-purpose TIMx timer features include:

- 16-bit up, down, up/down auto-reload counter.
- 16-bit programmable prescaler used to divide (also “on the fly”) the counter clock frequency by any factor between 1 and 65536.
- Up to 4 independent channels for:
 - Input capture
 - Output compare
 - PWM generation (Edge- and Center-aligned modes)
 - One-pulse mode output
- Synchronization circuit to control the timer with external signals and to interconnect several timers.
- Interrupt/DMA generation on the following events:
 - Update: counter overflow/underflow, counter initialization (by software or internal/external trigger)
 - Trigger event (counter start, stop, initialization or count by internal/external trigger)
 - Input capture
 - Output compare
- Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes
- Trigger input for external clock or cycle-by-cycle current management

Figure 100. General-purpose timer block diagram



1

- + 15.3.1 Time-base unit
- 15.3.2 Counter modes
 - Upcounting mode
 - Downcounting mode
 - Center-aligned mode (up/down counting)

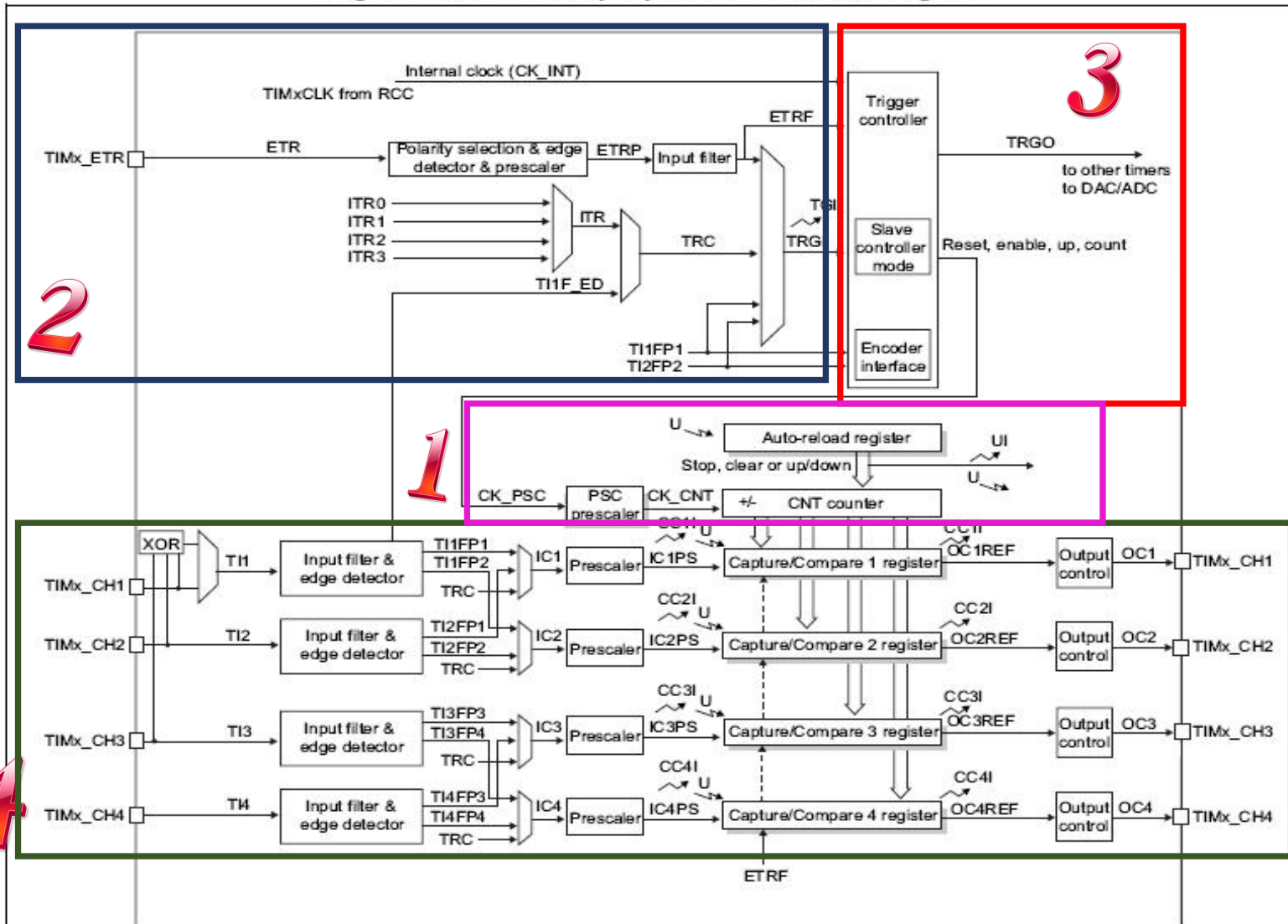
2

- 15.3.3 Clock selection
 - Internal clock source (CK_INT)
 - External clock source mode 1
 - External clock source mode 2

4

- + 15.3.4 Capture/compare channels
- + 15.3.5 Input capture mode
- + 15.3.6 PWM input mode
- + 15.3.7 Forced output mode
- + 15.3.8 Output compare mode
- + 15.3.9 PWM mode
- + 15.3.10 One-pulse mode
- + 15.3.11 Clearing the OCxREF signal on an external event
- + 15.3.12 Encoder interface mode
- + 15.3.13 Timer input XOR function
- + 15.3.14 Timers and external trigger synchronization
- + 15.3.15 Timer synchronization
- + 15.3.16 Debug mode

Figure 100. General-purpose timer block diagram



15.4 TIMx registers

15.4.1 TIMx control register 1 (TIMx_CR1)

15.4.2 TIMx control register 2 (TIMx_CR2)

15.4.3 TIMx slave mode control register (TIMx_SMCR)

15.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)

15.4.5 TIMx status register (TIMx_SR)

15.4.6 TIMx event generation register (TIMx_EGR)

15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

15.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)

15.4.9 TIMx capture/compare enable register (TIMx_CCER)

15.4.10 TIMx counter (TIMx_CNT)

15.4.11 TIMx prescaler (TIMx_PSC)

15.4.12 TIMx auto-reload register (TIMx_ARR)

15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

15.4.14 TIMx capture/compare register 2 (TIMx_CCR2)

15.4.15 TIMx capture/compare register 3 (TIMx_CCR3)

15.4.16 TIMx capture/compare register 4 (TIMx_CCR4)

15.4.17 TIMx DMA control register (TIMx_DCR)

15.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

Part IV : General Timer

--Timing & Counting (Clock Selection)


```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct)
```

Time-base unit

The time-base unit includes:

- Counter register (TIMx_CNT)
- Prescaler register (TIMx_PSC)
- Auto-Reload register (TIMx_ARR)

Counting mode

DIR

Clock source

15.4 TIMx registers

- 15.4.1 TIMx control register 1 (TIMx_CR1)
- 15.4.2 TIMx control register 2 (TIMx_CR2)
- + 15.4.3 TIMx slave mode control register (TIMx_SMCR)
- 15.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)
- 15.4.5 TIMx status register (TIMx_SR)
- 15.4.6 TIMx event generation register (TIMx_EGR)
- + 15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)
- + 15.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)
- + 15.4.9 TIMx capture/compare enable register (TIMx_CCER)
- 15.4.10 TIMx counter (TIMx_CNT)
- 15.4.11 TIMx prescaler (TIMx_PSC)
- 15.4.12 TIMx auto-reload register (TIMx_ARR)
- 15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)
- 15.4.14 TIMx capture/compare register 2 (TIMx_CCR2)
- 15.4.15 TIMx capture/compare register 3 (TIMx_CCR3)
- 15.4.16 TIMx capture/compare register 4 (TIMx_CCR4)
- 15.4.17 TIMx DMA control register (TIMx_DCR)
- + 15.4.18 TIMx DMA address for full transfer (TIMx_DMAR)



Internal clock source (CK_INT)

If the slave mode controller is disabled (SMS=000 in the TIMx_SMCR register), then the CEN, DIR (in the TIMx_CR1 register) and UG bits (in the TIMx_EGR register) are actual control bits and can be changed only by software (except UG which remains cleared automatically). As soon as the CEN bit is written to 1, the prescaler is clocked by the internal clock CK_INT.

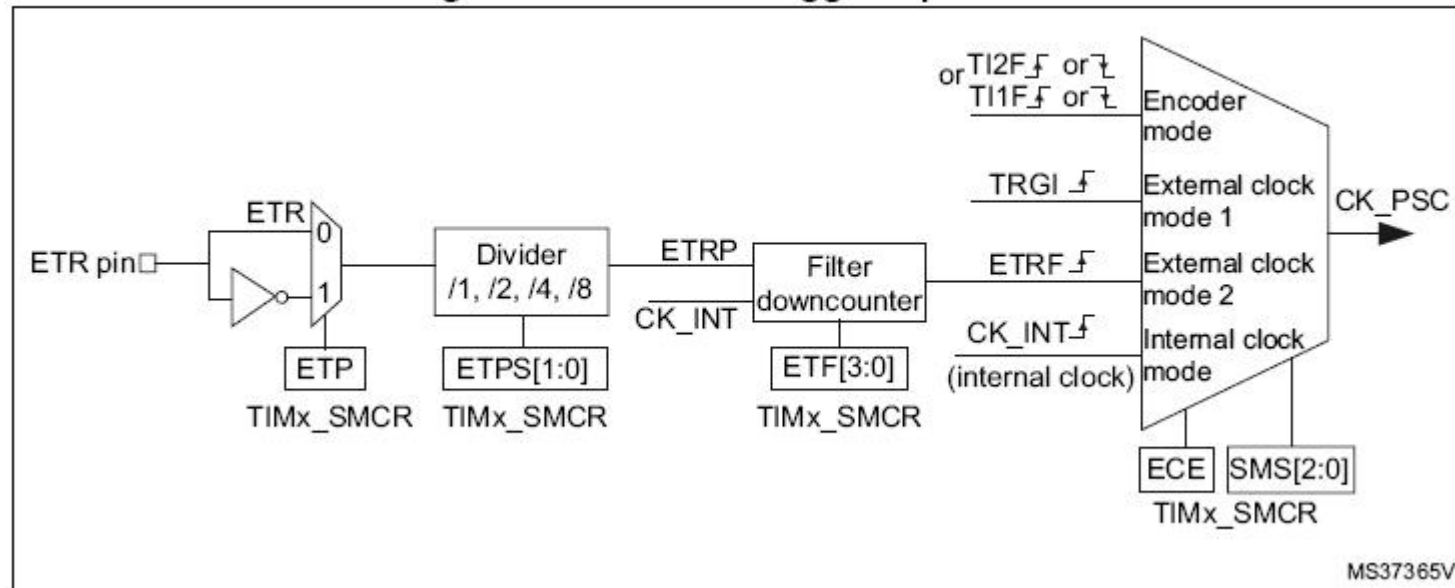
```
void TIM_InternalClockConfig(TIM_TypeDef* TIMx);
```

External clock source mode 2

This mode is selected by writing ECE=1 in the TIMx_SMCR register.

The counter can count at each rising or falling edge on the external trigger input ETR.

Figure 123. External trigger input block



```
void TIM_ETRClockMode2Config(TIM_TypeDef* TIMx, uint16_t TIM_ExtTRGPrescaler,  
uint16_t TIM_ExtTRGPolarity, uint16_t ExtTRGFilter);
```



```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct)
```

Time-base unit

The time-base unit includes:

- Counter register (TIMx_CNT)
- Prescaler register (TIMx_PSC)
- Auto-Reload register (TIMx_ARR)

Counting mode

DIR

Clock source

```
void TIM_InternalClockConfig(TIM_TypeDef* TIMx);  
void TIM_ETRClockMode2Config(TIM_TypeDef* TIMx, uint16_t TIM_ExtTRGPrescaler,  
uint16_t TIM_ExtTRGPolarity, uint16_t ExtTRGFilter);
```

15.4 TIMx registers

15.4.1 TIMx control register 1 (TIMx_CR1)

15.4.2 TIMx control register 2 (TIMx_CR2)

15.4.3 TIMx slave mode control register (TIMx_SMCR)

15.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)

15.4.5 TIMx status register (TIMx_SR)

15.4.6 TIMx event generation register (TIMx_EGR)

15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

15.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)

15.4.9 TIMx capture/compare enable register (TIMx_CCER)

15.4.10 TIMx counter (TIMx_CNT)

15.4.11 TIMx prescaler (TIMx_PSC)

15.4.12 TIMx auto-reload register (TIMx_ARR)

15.4.16 TIMx capture/compare register 4 (TIMx_CCR4)

15.4.17 TIMx DMA control register (TIMx_DCR)

15.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

General timer--Timing Configuration (CK_INT)

1、 Enable timer clock

```
RCC_APB1PeriphClockCmd();
```

2、 Timer Clock Configuration

```
TIM_DeInit();
```

```
TIM_InternalClockConfig();
```

**CK_INT , default setting ,
can be omitted**

3、 Initialization the time base structure

```
TIM_TimeBaseInit();
```

4、 Timer interrupt configuration

NVIC Configuration

```
TIM_ClearFlag();
```

```
void TIM_ITConfig();
```

5、 Enable timer

```
TIM_Cmd();
```

6、 Write interrupt service function.

```
TIMx_IRQHandler();
```

0x4000 4000 - 0x4000 43FF	Reserved	APB1
0x4000 3C00 - 0x4000 3FFF	SPI3/I2S	
0x4000 3800 - 0x4000 3BFF	SPI2/I2S	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 2400 - 0x4000 27FF	Reserved	
0x4000 2000 - 0x4000 23FF	TIM14 timer	
0x4000 1C00 - 0x4000 1FFF	TIM13 timer	
0x4000 1800 - 0x4000 1BFF	TIM12 timer	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	TIM5 timer	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	



```
void TIM_TimeBaseInit(TIM_TypeDef* TIMx, TIM_TimeBaseInitTypeDef* TIM_TimeBaseInitStruct)
```

```
51 | typedef struct
52 | {
53 |     uint16_t TIM_Prescaler;      /*!< Specifies the prescaler value used to divide the TIM clock.
54 |                                   This parameter can be a number between 0x0000 and 0xFFFF */
55 |
56 |     uint16_t TIM_CounterMode;    /*!< Specifies the counter mode.
57 |                                   This parameter can be a value of @ref TIM_Counter_Mode */
58 |
59 |     uint16_t TIM_Period;         /*!< Specifies the period value to be loaded into the active
60 |                                   Auto-Reload Register at the next update event.
61 |                                   This parameter must be a number between 0x0000 and 0xFFFF. */
62 |
63 |     uint16_t TIM_ClockDivision; /*!< Specifies the clock division.
64 |                                   This parameter can be a value of @ref TIM_Clock_Division_CKD */
65 |
66 |     uint8_t TIM_RepetitionCounter; /*!< Specifies the repetition counter value. Each time the RCR downcounter
67 |                                   reaches zero, an update event is generated and counting restarts
68 |                                   from the RCR value (N).
69 |                                   This means in PWM mode that (N+1) corresponds to:
70 |                                   - the number of PWM periods in edge-aligned mode
71 |                                   - the number of half PWM period in center-aligned mode
72 |                                   This parameter must be a number between 0x00 and 0xFF.
73 |                                   @note This parameter is valid only for TIM1 and TIM8. */
74 | } TIM_TimeBaseInitTypeDef;
75 |
```

```
void TIM_Cmd(TIM_TypeDef* TIMx, FunctionalState NewState);
```

```
FlagStatus TIM_GetFlagStatus(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

```
void TIM_ClearFlag(TIM_TypeDef* TIMx, uint16_t TIM_FLAG);
```

```
void TIM_ITConfig(TIM_TypeDef* TIMx, uint16_t TIM_IT, FunctionalState NewState);
```

```
ITStatus TIM_GetITStatus(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

```
void TIM_ClearITPendingBit(TIM_TypeDef* TIMx, uint16_t TIM_IT);
```

General timer--Counting Configuration(ETRClockMode2)

1、 Enable timer clock

```
RCC_APB1PeriphClockCmd();
```

2、 Timer Clock Configuration

```
TIM_DeInit();  
TIM_ETRClockMode2Config();
```

3、 Initialization the time base structure

```
TIM_TimeBaseInit();
```

4、 Set Counter to be 0

```
TIM_SetCount();
```

5、 Enable timer

```
TIM_Cmd();
```

```
TIM_GPIO_Config()  
GPIO_Mode_IN_FLOATING
```

0x4000 4000 - 0x4000 43FF	Reserved	APB1
0x4000 3C00 - 0x4000 3FFF	SPI3/I2S	
0x4000 3800 - 0x4000 3BFF	SPI2/I2S	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 2400 - 0x4000 27FF	Reserved	
0x4000 2000 - 0x4000 23FF	TIM14 timer	
0x4000 1C00 - 0x4000 1FFF	TIM13 timer	
0x4000 1800 - 0x4000 1BFF	TIM12 timer	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	TIM5 timer	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	

Detect the Frequency of a Square wave input

```
TIM3 : TIM_ETRClockMode2Config(TIM3, TIM_ExtTRGPSC_OFF, TIM_ExtTRGPolarity_Inverted,0);
```

```
TIM2 : TIM_InternalClockConfig(TIM2); Timing 200ms
```

1：利用ETR输入方波作为触发信号，TIM3--PD2引脚，然后计数；

2：另外一个定时器作定时，也就是闸门时间。

在闸门时间内检测输入脉冲的个数；闸门时间越长，检测精度越高，但是实时性越差；此处简化编程难度，闸门时间为200ms，待检测信号频率大于50HZ；

表 32-1 高级控制和通用定时器通道引脚分布

	高级定时器		通用定时器			
	TIM1	TIM8	TIM2	TIM5	TIM3	TIM4
CH1	PA8/PE9	PC6	PA0/PA15	PA0	PA6/PC6/PB4	PB6/PD12
CH1N	PB13/PA7/PE8	PA7				
CH2	PA9/PE11	PC7	PA1/PB3	PA1	PA7/PC7/PB5	PB7/PD13
CH2N	PB14/PB0/PE10	PB0				
CH3	PA10/PE13	PC8	PA2/PB10	PA2	PB0/PC8	PB8/PD14
CH3N	PB15/PB1/PE12	PB1				
CH4	PA11/PE14	PC9	PA3/PB11	PA3	PB1/PC9	PB9/PD15
ETR	PA12/PE7	PA0	PA0/PA15		PD2	PE0
BKIN	PB12/PA6/PE15	PA6				

Homework6-2

- Complete the project that is to measure the frequency of the input signal based on the unfinished project on BB. Sort out the the code flow chart.

Attention ! !

输入信号哪里来？ 另外一个同学用基本定时器写了一个方波输出，两个人换着测。不要用信号源 ! ! !

SSCOM Setting



 CH340驱动(USB串口驱动).zip



作业说明

平时的作业是一部分分数，每次总分100；

最后大家来提交学习板的时候，我会随机从以往作业中挑选一个并做一下参数修改，现场演示；相当于一次大作业，总分200；